

Business problem (RCA)

Deliveries are often arriving **after the promised date**, which leads to **bad reviews** and **fewer repeat purchases**. We don't have clear visibility into **where the delays happen** (before shipping vs during transit) or **which routes and sellers** are causing most of the late orders.

What we want (outcomes)

1. **See the truth in one place:** a single, clean view of each shipment with when it was approved, when it was shipped, when it was delivered, and the promised date.
 2. **Find the worst routes ("lanes"):** which **seller_state** → **customer_state** routes fail most often.
 3. **Spot slow sellers:** who takes too long **before** shipping (handling time).
 4. **Separate the cause:** is lateness mainly **handling** (seller delay) or **transit** (courier delay) on each route?
 5. **List the hot spots:** the few **seller and lane** combinations that cause most late orders.
-

Why we're doing this (business value)

- **Customer experience (CX):** fewer 1–2★ reviews → better trust → more repeat buyers.
- **Operational focus:** instead of boiling the ocean, we target **specific lanes and sellers**.
- **Speed:** we stop re-doing messy joins every time by building one reusable base table.
- **Accountability:** sellers and couriers know exactly what to improve (handling vs transit).

Data we'll use (Olist tables)

- `olist_orders` (timestamps, status, promise date)
 - `olist_order_items` (links orders ↔ sellers, item prices)
 - `olist_customers`, `olist_sellers` (states/cities)
 - `olist_order_reviews` (ratings for CX check)
 - `olist_products`, `product_category_name_translation` — only if we want category cuts later
-

Key definitions

- **Lane:** `seller_state` → `customer_state` (a route).
 - **Handling time:** approval → hand-off to carrier (seller side).
 - **Transit time:** hand-off to carrier → delivered to customer (courier side).
 - **On-time:** delivered **on or before** the estimated delivery date.
 - **Late:** delivered **after** the estimate.
-

Specific tasks (root-cause plan)

Task 1 — Build the “delivery facts” base

Goal: One clean table/view with one row per **order and seller** and these fields: `seller_state`, `customer_state`, `approved_at`, `shipped_at`, `delivered_at`, `promised_at`, `handling_days`, `transit_days`, `lead_time_days`, `is_on_time`.

Why: we’ll reuse this for every question; it avoids repeated, error-prone joins.

Output: a saved view (e.g., `v_delivery_facts_plus`).

Tables: `orders`, `order_items`, `customers`, `sellers`.

Task 2 — Find the worst lanes (routes)

Goal: For each lane, compute **orders**, **on_time_rate**, **average lead time**, and **typical handling/transit** (use median to ignore outliers).

Why: this shows which routes actually need attention first.

Output: a short list of **10 worst lanes** with enough volume (e.g., ≥ 300 orders).

Tables: the base view from Task 1.

Task 3 — Find slow-handling sellers

Goal: For each seller, measure **median handling time** and **on_time_rate** (with a fair volume cut, e.g., ≥ 200 delivered orders).

Why: highlights seller-side operational issues before the package even ships.

Output: top 20 **slow-handling sellers** with volumes and on-time rates.

Tables: the base view from Task 1.

Task 4 — Diagnose each lane: handling vs transit

Goal: For late orders **within each lane**, decide whether lateness is mostly **handling-driven** or **transit-driven**. Use that lane's own **p75 thresholds** (75th percentile) for handling and transit to label the driver.

Why: tells ops *what to fix* on that route—coach sellers (handling) or push the courier (transit).

Output: a lane table with counts and % of late orders due to **handling** vs **transit**, plus a “primary_driver” column.

Tables: the base view from Task 1.

Task 5 — Surface hot spots (seller × lane)

Goal: Find **seller × lane** pairs with **high late rate** and **enough orders** (e.g., ≥ 50).

Why: most of the pain comes from a few concentrated pairs—this becomes the **action list** for tomorrow's call.

Output: top ~20 seller × lane rows with orders, late_rate, median handling/transit to guide the fix.

Tables: the base view from Task 1.
